SYSTEM SOFTWARE
PAGING AND SEGMENTATION

A student-friendly guide to memory management techniques

Concepts • Address translation • Page tables • Segmentation tables • Worked examples • Comparison

Clear explanations, diagrams in text form, formulas and practice questions
22-page study notes

1. Introduction to Memory Management

Memory management is an important function of an operating system. It manages main memory (RAM) and decides how processes are placed in memory while they execute.

Why is memory management needed?

• Several processes may need memory at the same time.

• The operating system must allocate memory to processes and reclaim it when they finish.

• A process should not access memory belonging to another process without permission.

• Memory should be used efficiently while supporting process execution.

Logical and physical address

A logical address (also called a virtual address) is generated by the CPU while a program runs. A physical address identifies a location in main memory (RAM). The Memory Management Unit (MMU) translates logical addresses into physical addresses.

Two memory management techniques

• Paging: divides a process's logical memory into fixed-size pages and physical memory into equal-size frames.

• Segmentation: divides a program into logical, variable-size units such as code, data and stack.

Both techniques allow a process to be placed in non-contiguous areas of physical memory.

2. Contiguous Memory Allocation and Its Problems

In contiguous memory allocation, each process is placed in one continuous block of physical memory. This method is simple, but it can create memory-use problems.

Fragmentation

Fragmentation means that memory space is wasted or becomes difficult to use efficiently.

External fragmentation

External fragmentation occurs when free memory is split into many small holes. The total free memory may be large enough for a request, but no single free block is large enough.

```
Free blocks:  10 KB | 5 KB | 8 KB
New request:          18 KB

Total free memory = 23 KB
Largest free block = 10 KB
The 18 KB request cannot fit in one hole.
```

Internal fragmentation

Internal fragmentation occurs when an allocated block is larger than the memory actually requested. The unused space inside the allocated block is wasted.

Paging avoids external fragmentation in the allocation of frames, but may cause internal fragmentation in the last page. Segmentation can suffer from external fragmentation because segments have variable sizes.

3. Paging: Basic Concept

Paging is a memory management technique in which the logical address space of a process is divided into fixed-size blocks called pages. Physical memory is divided into blocks of the same size called frames.

Important terms

• Page: a fixed-size block of a process's logical memory.

• Frame: a fixed-size block of physical memory.

• Page size: the size of each page.

• Frame size: the size of each frame; it is equal to page size.

• Page table: a data structure that records which frame contains each page.

How paging works

• The CPU generates a logical address.

• The address is separated into a page number and an offset.

• The page number is used to find the corresponding frame number in the page table.

• The frame number is combined with the unchanged offset to form the physical address.

A process's pages do not need to occupy consecutive frames. This is one of the major advantages of paging.

4. Paging: Logical and Physical Address

A logical address in a paging system contains two parts: page number (p) and page offset (d). The page number identifies a page; the offset identifies a byte or location within that page.

```
Logical address
+----------------------+----------------------+
|      Page number p   |      Offset d        |
+----------------------+----------------------+

Page table:
Page number  ->  Frame number

Physical address:
+----------------------+----------------------+
|     Frame number     |      Offset d        |
+----------------------+----------------------+
```

Address translation formula

If page size is S and logical address is LA:

```
Page number p = floor(LA / S)
Offset d       = LA mod S

Physical address PA = (frame number x S) + d
```

The offset does not change during translation because the byte's position within the page is the same within its physical frame.

5. Paging: Worked Numerical Example

Assume a system has a page size of 1,024 bytes. A process generates logical address 2,500. The page table says that page 2 is stored in frame 7.

Step 1: Find the page number

```
p = floor(2500 / 1024)
  = 2
```

Step 2: Find the offset

```
d = 2500 mod 1024
  = 452
```

Step 3: Find the physical address

```
PA = (frame number x page size) + offset
   = (7 x 1024) + 452
   = 7168 + 452
   = 7620 bytes
```

Answer: logical address 2,500 maps to physical address 7,620, based on the given page table.

Always check that the offset is smaller than the page size. Here, 452 is less than 1,024.

6. Paging: Page Table

A page table is maintained for each process. It maps the process's page numbers to physical frame numbers.

Page number	Frame number
0	4
1	1
2	7
3	5

For example, page 2 is stored in frame 7. The page table lets the operating system place pages in any available frames.

What may a page table entry contain?

• Frame number for the page.

• Valid or present bit to indicate whether the page is currently in memory.

• Protection bits to control read, write or execute access.

• Other status information, such as reference or modified (dirty) bits.

The exact fields depend on the processor architecture and operating system.

7. Paging: Page Table Size and Address Bits

Page tables need memory. Their size depends on the number of pages in a process and the size of each page-table entry.

Number of pages

```
Number of pages = Logical address space / Page size

Example:
Logical address space = 16 KB
Page size = 1 KB

Number of pages = 16 KB / 1 KB = 16 pages
```

Address bits

For a page size that is a power of two, the number of offset bits is log2(page size in bytes). For example, a 4 KB page has 12 offset bits because 4 KB = 4,096 bytes = 2^12.

```
If logical address size = 16 bits
Page size = 1 KB = 2^10 bytes

Offset bits = 10
Page number bits = 16 - 10 = 6
Number of pages = 2^6 = 64
```

A larger page size reduces the number of pages and often the size of the page table, but may increase internal fragmentation.

8. Translation Lookaside Buffer (TLB)

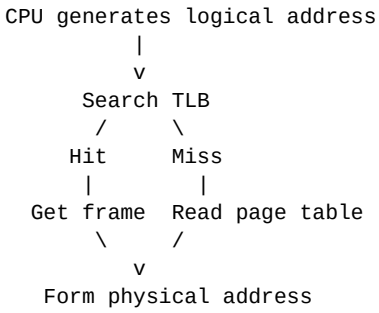
A Translation Lookaside Buffer (TLB) is a small, fast cache that stores recently used page-table entries. It helps reduce the time required for address translation.

TLB hit

A TLB hit occurs when the required page-table entry is found in the TLB. The frame number can be obtained quickly.

TLB miss

A TLB miss occurs when the required entry is not in the TLB. The system must consult the page table in memory and may then place the entry in the TLB.



A TLB miss is not the same as a page fault. A TLB miss means the translation is not cached; the page may still be present in RAM.

9. Paging: Effective Access Time

Effective Access Time (EAT) is the average time required to access memory, taking TLB hits and misses into account.

Simple model

Assume a TLB lookup takes t time units, a memory access takes m time units, and the TLB hit ratio is h. If a TLB hit needs one memory access and a miss needs two memory accesses (one for the page table and one for the data), then:

```
EAT = h(t + m) + (1 - h)(t + 2m)
```

Example

Let TLB lookup time be 10 ns, memory access time be 100 ns, and hit ratio be 0.90.

```
EAT = 0.90(10 + 100) + 0.10(10 + 200)
    = 0.90(110) + 0.10(210)
    = 99 + 21
    = 120 ns
```

This is a simplified model. Actual systems may have different hardware arrangements, caches and page-table structures.

10. Paging: Protection and Sharing

Memory protection

Paging can support memory protection through bits stored in page-table entries. These bits can specify whether a page may be read, written or executed.

• Read permission: allows data to be read.

• Write permission: allows data to be modified.

• Execute permission: allows instructions to be fetched and run.

• Valid/present information: helps detect invalid references or pages not currently resident.

Page sharing

Two or more processes can sometimes map their virtual pages to the same physical frame. This can be useful for shared libraries or shared memory.

Shared code can be mapped read-only so processes can execute it without modifying the common copy. Writable shared memory requires appropriate permissions and coordination.

Why protection matters

Protection helps prevent accidental or unauthorised access across process boundaries. The operating system and processor hardware work together to enforce these rules.

11. Demand Paging and Page Fault

Demand paging is a virtual memory technique in which a page is brought into main memory only when it is needed.

Page fault

A page fault occurs when a process refers to a page that is not currently in physical memory. The operating system checks whether the reference is valid and, if it is, arranges to load the page.

Typical page-fault handling steps

• The CPU detects that the page is not present and traps to the operating system.

• The operating system checks whether the memory reference is valid.

• It locates the required page in secondary storage.

• A free frame is selected, or a page-replacement decision is made.

• The required page is read into the selected frame.

• The page table is updated and the interrupted instruction is restarted.

A page fault is a normal event in demand-paged virtual memory, although handling it is much slower than accessing a page already in RAM.

12. Page Replacement Algorithms

If a page fault occurs and there is no free frame, the operating system may need to remove a page from memory. This is called page replacement.

FIFO (First-In, First-Out)

FIFO removes the page that has been in memory for the longest time. It is simple to implement using a queue, but it may remove a frequently used page.

Optimal page replacement

The optimal algorithm removes the page whose next use is farthest in the future. It gives the lowest possible number of page faults for a known reference string, but future references are not normally known in advance. It is mainly used as a benchmark.

LRU (Least Recently Used)

LRU replaces the page that has not been used for the longest time in the past. It uses recent history as an estimate of future use.

The choice of algorithm affects the number of page faults and system performance.

13. Page Replacement: Small Example

Consider three available frames and the reference string: 1, 2, 3, 1, 4. Assume FIFO replacement.

```
Reference: 1   2   3   1   4
Frames:
Step 1:   1   -   -    Fault
Step 2:   1   2   -    Fault
Step 3:   1   2   3    Fault
Step 4:   1   2   3    Hit
Step 5:   4   2   3    Fault
```

At step 5, page 1 is the oldest page in memory, so FIFO removes page 1 and loads page 4.

Count

```
Page faults = 4
Page hits   = 1
```

When solving page-replacement problems, write the reference string clearly, track the frame contents after every reference, and count a fault only when the required page is absent.

14. Paging: Advantages and Disadvantages

Advantages

• Eliminates external fragmentation in physical frame allocation.

• A process can be stored in non-contiguous physical frames.

• Supports virtual memory and demand paging.

• Makes memory allocation easier because all frames have the same size.

• Can provide page-level protection and sharing.

Disadvantages

• May cause internal fragmentation, especially in the final page.

• Page tables require additional memory.

• Address translation adds overhead; a TLB helps reduce it.

• Page faults can significantly slow execution.

• Managing page tables and replacement requires operating-system support.

Paging is widely used because it provides flexible memory allocation and works well with virtual memory.

15. Segmentation: Basic Concept

Segmentation is a memory management technique that divides a program into logical, variable-size parts called segments. Each segment represents a meaningful unit of a program.

Examples of segments

• Code segment: contains program instructions.

• Data segment: contains global and static data.

• Stack segment: stores function calls and local data.

• Heap segment: supports dynamic memory allocation.

Unlike pages, segments do not all have the same size. Their sizes depend on the program's logical organisation.

Logical address in segmentation

A logical address consists of two parts: segment number (s) and offset (d). The segment number identifies the segment, and the offset identifies a location within that segment.

```
Logical address
+----------------------+----------------------+
|     Segment number s |       Offset d       |
+----------------------+----------------------+
```

16. Segmentation Table

A segment table stores information about each segment. Two important fields are the base and the limit.

• Base: starting physical address of the segment.

• Limit: length or size of the segment.

Segment	Base	Limit
0	1000	400
1	2400	600
2	5000	300

For segment 1, the segment starts at physical address 2,400 and has a length of 600 addressable units. Valid offsets are from 0 through 599.

Why use a limit?

The limit prevents a process from referring beyond the end of a segment. The offset must be smaller than the segment's limit.

17. Segmentation: Address Translation

To translate a logical address (s, d), the system uses the segment number to find the matching segment-table entry.

Translation steps

• Check whether the segment number is within the segment table.

• Read the segment's base and limit.

• Check whether the offset d is less than the limit.

• If valid, add the offset to the base to get the physical address.

• If invalid, raise a protection fault or trap.

```
If d < limit:
Physical address = base + d

If d >= limit:
Invalid address reference
```

Worked example

Suppose segment 1 has base 2,400 and limit 600. The logical address is (1, 250).

```
Offset 250 < limit 600, so it is valid.

Physical address = 2400 + 250
                 = 2650
```

If the offset were 650, it would be invalid because 650 is not less than 600.

18. Segmentation: Another Numerical Example

Consider the following segment table. Find the physical address for logical address (2, 120).

Segment	Base	Limit
0	4000	500
1	7000	800
2	9000	250

Solution

```
Segment number = 2
Base = 9000
Limit = 250
Offset = 120

Check: 120 < 250  -> Valid

Physical address = 9000 + 120
                 = 9120
```

Answer: 9,120.

Try this

For segment 0, find the physical address for offset 520. Since the limit is 500, the offset is outside the segment. The reference is invalid and no physical address is generated.

19. Segmentation: Fragmentation, Protection and Sharing

External fragmentation

Because segments have variable sizes, free memory can become divided into holes of different sizes. A new segment may not fit even if the total free memory is sufficient. This is external fragmentation.

Compaction

Compaction is a technique that moves allocated segments so scattered free spaces can be combined into a larger free block. It can be costly because data must be moved and addresses may need updating.

Protection

Each segment can have its own access permissions. For example, a code segment may be read-only and executable, while a data segment may be readable and writable.

Sharing

A segment can represent a logical unit, such as shared code or a library. Sharing can be arranged by mapping the same physical region into multiple processes, subject to protection rules.

20. Paging vs Segmentation

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments
View	Physical-memory management	Logical program units
Logical address	Page number + offset	Segment number + offset
Table entry	Frame number and status bits	Base and limit
Fragmentation	Internal fragmentation possible	External fragmentation possible
Compaction	Not normally needed for frames	May be used
Program view	Usually transparent to programmer	Reflects logical program structure
Protection	Can be applied per page	Can be applied per segment

Main difference to remember

Paging divides memory into equal-sized blocks. Segmentation divides a program into meaningful parts whose sizes can differ.

21. Combined Schemes and Quick Revision

Paged segmentation

Some systems combine segmentation and paging. A program is first divided into logical segments, and each segment is then divided into fixed-size pages. A logical address may identify a segment, a page within that segment, and an offset within the page.

• Segmentation gives a logical view of a program.

• Paging allows the pages of each segment to be placed in available frames.

• The design combines logical organisation with fixed-size physical allocation.

Important formulas

```
PAGING
Page number = floor(Logical address / Page size)
Offset      = Logical address mod Page size
Physical address = (Frame number x Page size) + Offset

SEGMENTATION
Check: Offset < Segment limit
Physical address = Segment base + Offset
```

Quick revision questions

• What is the difference between a page and a frame?

• What are the two parts of a logical address in paging?

• What is the role of a page table?

• Differentiate a TLB miss and a page fault.

• What is external fragmentation?

• What are base and limit in a segment table?

• Why is an offset checked against the segment limit?

• Give two differences between paging and segmentation.

Conclusion

Paging and segmentation are important memory management techniques. Paging uses fixed-size blocks and page tables to map pages to frames. Segmentation uses variable-size logical units and a segment table containing base and limit values. Understanding address translation and fragmentation is essential for solving numerical problems in System Software.